Lambda v2

Microarchitecture Dataflow Walkthrough

Longhorn Silicon LLM Inference Accelerator

$4\,\mathrm{mm}^2$ Standalone Transformer Accelerator — TSMC N16FFC

Version: `lambda-v2-dataflow-0.3` (architecture draft)
Date: 2026-04-27
Status: Pre-tape-out reference; companion to `Lambda_v2_4mm2.yaml`

Authors: Chaithu Talasila and Alan Schwartz, The University of Texas at Austin

Reference: `LonghornSilicon/architecture`

Scope. This document is a unit-by-unit walkthrough of the eight functional blocks on the Lambda v2 die, traced in the order data flows during a single decode token. Each block is introduced concretely as it appears on the live execution path of Llama-3.2-3B generating token #42. Numbers are validated against `scripts/v2_design_space/lambda_v2_audit.py`; companion documents are the machine-readable spec `Lambda_v2_4mm2.yaml`, the visual floorplan `Lambda_v2_floorplan.html`, and the v1-vs-v2 decision record `STATUS.md`.

Contents

1 Execution Context **3**
1.1 Block Inventory . . . 3

2 Stage 0 — Boot and Weight DMA (HIF) **3**

3 Stage 1 — Layer Schedule Dispatch (LSU) **3**

4 Stage 2 — Address Translation and DRAM Issue (MSC) **4**
4.1 Block-Table TLB . . . 4
4.2 DRAM Command Sequencing . . . 4

5 Stage 3 — Mixed-Signal Bus Driving (LPDDR5X PHY) **4**

6 Stage 4 — Weight Staging and SRAM Banks **5**

7 Stage 5 — QKV Projection (MatE) **6**

8 Stage 6 — Rotary Positional Embedding (VecU) **6**

9 Stage 7 — KV Compression via TurboQuant (KCE) **7**
9.1 Walsh-Hadamard Transform . . . 7
9.2 Lloyd-Max Codebook . . . 7

10 Stage 8 — Compressed-Domain Q·K$^\top$ Scoring (MatE) **8**

11 Stage 9 — Online FlashAttention-3 Softmax (VecU) **8**

12 Stage 10 — Tiled Softmax·V Accumulation (MatE + VecU) **9**

13 Stage 11 — Feed-Forward Network (MatE + VecU) **9**

14 Stage 12 — RMSNorm and Residual (VecU) **10**

15 Stage 13 — LM Head, Sampling, and Emission **10**

16 Complete Chip Dataflow **10**

17 Performance Summary **11**

1. Execution Context

The user has been chatting for several turns; the KV cache is partially populated with prior-turn context. The chip is about to generate token #42. Model: Llama-3.2-3B (3,072-dim hidden state, 28 transformer layers, 8 KV heads of dimension 128). Sections that follow describe one discrete stage of hardware activity each, in execution order. Where multiple blocks operate concurrently, the ordering reflects the logical dependency graph rather than strict serialization.

1.1 Block Inventory

Block	Full Name	Area (mm^2)	Primary Role
HIF	Host Interface	0.30	USB-C 2.0 endpoint; boot DMA, doorbells, JTAG
LSU	Layer Sequencer Unit	0.10	In-order RISC; 4 KB microcode; orchestration
MSC	Memory Subsystem Controller	0.18	LPDDR ctrl + 4-port SRAM xbar + 128-entry TLB
PHY	LPDDR5X ×16 PHY	1.20	Mixed-signal IP; 17/12 GB/s peak/sustained
MatE	Matrix Engine	0.10	8×8 INT8×INT4 systolic; 0.13 TOPS
VecU	Vector Unit	0.144	8-lane FP16/BF16 SIMD; transcendentals
KCE	KV Compression Engine	0.08	TurboQuant: Hadamard + Lloyd-Max codebook
SRAM	On-chip SRAM (4 banks)	0.71	0.8 MB: KV scratchpad + act buf + weight FIFO + ROM

Table 1: Lambda v2 functional blocks. Areas verified by `lambda_v2_audit.py`.

2. Stage 0 — Boot and Weight DMA (HIF)

When Lambda is plugged into a USB-C port, the **HIF (Host Interface)** is the first block to wake up. HIF is a USB-C 2.0 endpoint running at 480 Mbps, occupying ∼0.30 mm^2 at the die perimeter. At power-on it (1) enumerates as a USB device, (2) exposes a CSR window for driver register access, and (3) provides a JTAG and scan-chain debug port.

The host driver performs two operations over HIF. First, it writes ∼3,000 instructions of pre-compiled microcode into the LSU's instruction RAM — the static schedule for running Llama-3.2-3B. Second, it initiates a 1.5 GB DMA transfer of W4-quantized weights from host DRAM into the off-chip LPDDR5X package. At 60 MB/s sustained over USB-C 2.0, this transfer requires ∼25 seconds, but happens only once per power-on cycle. Weights persist in LPDDR5X until the device is unplugged.

Once armed, the chip waits for prompts via HIF's 16-deep doorbell queue. When the user submits a prompt, the host driver writes a doorbell entry; HIF asserts an interrupt; the LSU wakes and begins decode.

3. Stage 1 — Layer Schedule Dispatch (LSU)

The **LSU (Layer Sequencer Unit)** is the chip's control plane: a 3-stage in-order RISC pipeline (fetch → decode → dispatch) with 16 general-purpose registers and a 4 KB microcode RAM containing the entire pre-compiled execution schedule. Its minimalism is intentional — transformer decode is structurally identical across all 28 layers, so a static, deterministic program suffices. There is no branch predictor, no out-of-order execution, no cache hierarchy, no virtual memory.

The first instruction issued for token #42 is shown below.

```
LOAD_WEIGHTS  layer=0, slice=qkv_proj             ; DMA descriptor -> MSC queue
ISSUE_MAT_E   qkv_proj, in=act_buf, out=qkv_scratch ; stalls on weights
ISSUE_VEC_U   rope, q, k                          ; queued: runs after MatE
ISSUE_KCE_COMP k -> kv_scratchpad                 ; queued: runs after VecU
```

`LOAD_WEIGHTS` is not a CPU-style load — it writes a DMA descriptor into one of MSC's request queues and returns immediately. `ISSUE_MAT_E` hands off to the Matrix Engine, which stalls internally awaiting the incoming weight stream. The LSU continues speculatively, issuing subsequent instructions that queue behind the in-flight memory operation.

The absence of runtime control flow is not a limitation — it is the architectural claim. A pre-compiled schedule with zero branches eliminates an entire class of verification complexity. The LSU is the conductor; from this point forward, all real computation occurs in the functional units.

4. Stage 2 — Address Translation and DRAM Issue (MSC)

The **MSC (Memory Subsystem Controller)** receives the LSU's DMA descriptor and acts as the chip's memory traffic cop, binding together the SRAM crossbar, the LPDDR5X controller, the block-table TLB, and the DMA engine into a single arbitrated fabric.

4.1 Block-Table TLB

MSC first consults its 128-entry associative block-table TLB, which maps logical addresses of the form `(session_id, layer_id, block_id)` to physical SRAM bank offsets or DRAM row/column addresses. For layer-0 QKV weights, the mapping is static and pre-populated by the host driver at boot — a single-cycle TLB hit to a known DRAM address baked into the compiled schedule.

4.2 DRAM Command Sequencing

With the physical address resolved, MSC issues a DRAM read via the LPDDR5X controller. The controller serializes the request as a compliant LPDDR5X command sequence, honoring all timing parameters:

```
ACT  (bank_id, row_addr)  -> wait t_RCD  ~= 14 ns
READ (col_addr)            -> wait t_CCD  ~=  5 ns
READ (col_addr + stride)   -> wait t_CCD  ~=  5 ns
  ...                          (burst continues)
Data arrives at MSC read buffer ~30 ns after first READ.
```

The controller uses an open-page policy: rows activated for one access are held open for subsequent column reads from the same row, avoiding redundant ACT commands. Bank-conflict avoidance logic tracks which banks are open across concurrent requests. The 4-port SRAM crossbar remains idle during this initial fetch — all compute units are stalled pending first-weight arrival.

5. Stage 3 — Mixed-Signal Bus Driving (LPDDR5X PHY)

The **LPDDR5X ×16 PHY** is the only mixed-signal block on the die — licensed vendor IP from Synopsys DesignWare or Cadence Denali. At $\sim$1.20 mm^2 it is the largest single block and the primary area risk (R-Lv2-03). It sits at the die edge and exposes 16 data pads (DQ0–DQ15), 2

differential strobe pads (DQS/DQS#), differential clock pads (CK/CK#), and a command bus (CA0–CA6, CKE, CS).

When MSC issues an ACT command, the PHY's digital command path serializes it to the correct pin sequence and drives it across impedance-matched PCB traces to the off-chip LPDDR5X package. After t_{RCD} (∼14 ns), the row is open. On the READ command, the PHY's data receive path becomes active: its read-leveling logic uses the DQS strobes returning from the DRAM to time-align the 8,533 Mbps DQ bitstreams into the chip's 1 GHz synchronous clock domain via a continuous timing-recovery loop that compensates for temperature and voltage drift.

Performance bottleneck. 8,533 Mbps × 16 lanes / 8 bits/byte = 17 GB/s peak. After bank-conflict overhead and DRAM refresh, sustained throughput is ∼12 GB/s. Every gate of compute on the chip is fed through this single memory channel. Bandwidth — not arithmetic capacity — is the end-to-end performance constraint at every operating point.

Risk item R-Lv2-03. The PHY footprint is quoted at 1.20 mm^2 with a ±0.30 mm^2 real range (NDA-gated). At 1.0 mm^2 (best case): SRAM budget grows to 1.0 MB with margin. At 1.5 mm^2 (worst case): total die exceeds 4 mm^2 — requires downgrading to LPDDR4x ×16 (losing the 4–5B model class) or growing the die to 4.5 mm^2 (∼12% shuttle cost increase). Written quote required before floorplan freeze.

6. Stage 4 — Weight Staging and SRAM Banks

MSC writes the fetched weights into the `weight_stream_buffer` — a 50 KB single-port SRAM macro at the perimeter of MatE. This bank is deliberately undersized: at 12 GB/s with ∼100 ns first-byte latency, the minimum buffer depth needed to hide DRAM latency is 1.2 KB. At 50 KB, the buffer provides 40× that minimum, enabling double-buffering with headroom to warm-cache hot weight slices (e.g., the LM head, which is accessed on every token).

Bank	**Cap.**	**Ports**	**Area (mm^2)**	**Role**
`weight_stream_buffer`	50 KB	1-port	0.05	Weight FIFO; 40× DRAM-latency-hiding minimum
`activation_buffer`	0.3 MB	2-port	0.26	Layer input/output; MatE reads while VecU writes
`kv_scratchpad`	0.4 MB	1-port	0.35	Compressed K and V for live context
`codebook_const_rom`	64 KB	1-port RO	0.05	Lloyd-Max centroids; RoPE table; exp/rsqrt LUTs
Total	0.8 MB	—	0.71	—

Table 2: On-chip SRAM bank summary.

The KV scratchpad holds 50% of total SRAM capacity. This is an explicit architectural decision: long-context decode performance is KV-bandwidth-bound, and the weight buffer needs only enough depth to hide DRAM latency — not to match the KV working set. This asymmetry is the primary differentiator from classical weight-centric accelerators.

7. Stage 5 — QKV Projection (MatE)

The **MatE (Matrix Engine)** is an 8×8 array of Processing Elements (PEs), each containing one INT8×INT4 multiplier, one INT16 accumulator, and the latches required for weight-stationary dataflow. The array occupies 0.10 mm^2 of die — deliberately compact, because on a bandwidth-bound chip, compute area buys diminishing throughput.

```
       a0   a1   a2  ...  a7      <- activations stream right (INT8)
        |    |    |        |
W00 -> [PE]-[PE]-[PE]- ... -[PE]
        |    |    |        |
W10 -> [PE]-[PE]-[PE]- ... -[PE]    Each PE: acc += weight * activation
        |    |    |        |       Weight pinned for entire tile.
       ...  ...  ...      ...
        |    |    |        |
W70 -> [PE]-[PE]-[PE]- ... -[PE]
        |    |    |        |
       o0   o1   o2  ...  o7     <- INT16 partial sums drain from bottom
```

For the QKV projection of Llama-3.2-3B (3,072-dim hidden → 4,096-dim QKV combined), the schedule tiles the 3,072×4,096 GEMM into 8×8 chunks and streams them through the array. On each tile: 64 INT4 weight values latch into the 64 PEs (*one per PE* — this is weight-stationary). Then activations stream from the left edge in rows of 8 INT8 values per cycle, staggered by one cycle per row. Every PE multiplies its pinned weight by its arriving activation, accumulates the result into its INT16 register, and passes the activation rightward. Partial sums drain from the bottom edge.

Throughput. 64 PEs × 2 ops/cycle × 1 GHz = 128 GOPS peak. The 3,072-dim QKV projection on a single token requires ∼25M MAC operations, completing in ∼0.2 ms. Compute headroom over the bandwidth-bound floor is 1.6× constant — MatE idles waiting for the next weight batch at every operating point. Bandwidth, not arithmetic capacity, governs end-to-end throughput.

Output: `qkv_scratch` — three 3,072-element vectors (Q, K, V) written to `activation_buffer`.

8. Stage 6 — Rotary Positional Embedding (VecU)

LSU dispatches `ISSUE_VEC_U rope, q, k`. The **VecU (Vector Unit)** is the chip's programmable SIMD engine: 8 lanes operating on FP16 or BF16 scalars, sharing a 1 KB microcode RAM and 4 KB of transcendental lookup tables (exp, rsqrt, sigmoid — 64 entries each with linear interpolation logic). Every non-GEMM operation on the chip is implemented as a microcoded loop in VecU.

RoPE encodes token position into the query and key vectors by rotating consecutive element pairs by a frequency-dependent angle. For a 64-dimensional head, the head is partitioned into 32 pairs (x, y), and each pair undergoes the rotation:

$$(x, y) \;\rightarrow\; (x\cos\theta_k - y\sin\theta_k,\;\; x\sin\theta_k + y\cos\theta_k)$$

Angles θ_k are read from the RoPE frequency table in `codebook_const_rom`. VecU's microcode sequence for RoPE is ∼12 μops: load pair → load cos/sin from ROM → 4 multiplies + 2 adds → store. Eight lanes process 8 pairs in parallel, completing the 32 pairs per head in 4 microcode iterations. Both Q and K are rotated, for a total of ∼30 μops per token's RoPE application.

VecU's programmability eliminates four separate fixed-function blocks (RoPE, softmax, RMSNorm, SiLU) in favor of one verifiable SIMD engine. The microcode footprint — 1 KB — is negligible. The verification surface reduction is substantial. This was a deliberate area/complexity trade-off in the v2 architecture.

9. Stage 7 — KV Compression via TurboQuant (KCE)

LSU fires `ISSUE_KCE_COMP k -> kv_scratchpad`. The **KCE (KV Compression Engine)** is Lambda v2's primary IP contribution — the silicon implementation of TurboQuant. At 0.08 mm^2, it achieves 3.0× KV compression with no multiplication hardware whatsoever.

K is a 1,024-element FP16 vector (8 KV heads × 128 dim) for this token in this layer. KCE processes it in 16-element chunks. The pipeline for each chunk:

```
[16 x FP16 input elements]
              |
              v
16-pt Walsh-Hadamard Transform
  4 butterfly stages x 8 add/sub pairs = 32 add/sub
  Spreads outlier energy uniformly across coordinates
              |
              v
Nearest-Centroid Classifier
  7 comparators x 16 lanes = 112 comparators
  Selects nearest of 8 codebook entries (3-bit index)
              |
              v
Bit-Pack: 16 x 3-bit + 16-bit magnitude header = 64 bits
  FP16 256 bits -> 64 bits, ~3.0x effective compression
              |
              v
[Write 8 bytes -> kv_scratchpad]
```

9.1 Walsh-Hadamard Transform

The Hadamard butterfly is a specific arrangement of additions and subtractions that is mathematically equivalent to multiplying the input by an orthogonal ±1 rotation matrix. This rotation spreads outlier energy: if the input vector has one dominant coordinate (a common pathology in transformer KV caches), the butterfly redistributes that energy roughly uniformly across all 16 coordinates, producing a distribution that is approximately Beta-distributed — the target regime for the Lloyd-Max codebook.

9.2 Lloyd-Max Codebook

The codebook holds 8 centroids (64 bytes stored in `codebook_const_rom`), computed offline to be optimal for the Beta distribution. Each post-Hadamard coordinate independently selects the nearest centroid using a bank of 7 comparators. The resulting 3-bit index, combined with a 16-bit group-wise magnitude header, achieves ∼3 bits per element effective compression versus FP16's 16 bits per element — a 3.0× storage and bandwidth reduction.

Every operation in the KCE pipeline is an add, subtract, comparator evaluation, or ROM lookup.

There are no multipliers. This is why the entire block fits in 0.08 mm^2 and consumes only ∼50 mW at full throughput. The compressed K, and then V, are written to `kv_scratchpad` alongside all prior-token KV vectors for this layer.

10. Stage 8 — Compressed-Domain Q·K$^\top$ Scoring (MatE)

LSU dispatches `ISSUE_MAT_E qk_dot, q, k_compressed -> scores`. MatE switches from weight-stationary to output-stationary mode by asserting a single CSR mode bit, which changes which tensor is pinned in the PE array and which streams through.

In output-stationary mode, Q is pinned (8 query heads, one per row) and the compressed past-K vectors from `kv_scratchpad` stream through column by column. For a context of ∼3,000 accumulated tokens, this streams 3,000 compressed-K vectors through the array, producing a scores tensor of shape [3,000 × 8 query heads].

Compressed-domain dot product. For a symmetric quantization scheme, the dot product $\langle Q, \text{dequant}(K_c)\rangle$ is algebraically equivalent to $\text{scale} \cdot \langle Q, K_c\rangle$. The scale is a scalar applied once per row sum — not per multiply. MatE executes the full attention scoring in the compressed domain: INT8 Q × INT3 K multiplications with INT16 partial-sum accumulation. No FP16 dequantization step is required before scoring, and no intermediate FP16 K matrix is materialized.

11. Stage 9 — Online FlashAttention-3 Softmax (VecU)

LSU fires `ISSUE_VEC_U softmax_online, scores`. This is the most algorithmically significant microcode on the chip.

Naïve softmax requires materializing the full attention score matrix prior to computing the denominator $\exp(s - \max(s)) / \sum \exp(s)$. At 3,000 tokens × 8 heads × 2 bytes (FP16), that matrix occupies ∼50 KB — saturating the activation buffer. The FlashAttention-3 online algorithm avoids full materialization by processing scores in tiles and maintaining only two running scalars per attention row:

```
State per query row: (m_i, l_i, O_i)
    m_i  -- running maximum of all scores seen so far
    l_i  -- running sum of exp(score - m_i)
    O_i  -- running weighted value accumulator

On each new tile of scores (s_1, ..., s_T):
    m_new  = max(m_i, max(s_1...s_T))
    l_i   *= exp(m_i - m_new)                ; rescale for new max
    O_i   *= exp(m_i - m_new)                ; rescale accumulator
    m_i    = m_new
    l_i   += sum exp(s_j - m_new)
    O_i   += sum exp(s_j - m_new) * V_tile[j]

After final tile:  output = O_i / l_i        ; one division per row
```

The rescaling factor $\exp(m_{old} - m_{new})$ corrects for having used the wrong maximum in prior tiles — the algebra is exactly equivalent to full softmax. Intermediate storage per attention row is only (m_i, l_i): two scalars. No score tensor is ever fully materialized.

VecU's 8 lanes process 8 query-row states in parallel. The microcode loop is ∼32 μops per tile

(load tile → max-reduce → exp via LUT → multiply-accumulate → update state). Over 3,000-token context with tile size 32, ∼100 tiles run per layer, for ∼3,200 μops per softmax invocation. The exp lookup is a 64-entry ROM spanning $x \in [-16, 0]$ with linear interpolation to 0.05 ULP error.

12. Stage 10 — Tiled Softmax·V Accumulation (MatE + VecU)

In the FlashAttention-3 tiled algorithm, the `softmax_scores · V` accumulation is not a separate GEMM invocation — it is interleaved with the softmax tile loop within each iteration of Stage 9. On every tile:

1. MatE scores $Q \cdot K^\top$ for the current tile of past-K vectors from `kv_scratchpad`.
2. VecU updates the running state (m_i, l_i) and rescales O_i.
3. VecU accumulates $O_i \mathrel{+}= \text{softmax_weight}_j \cdot V_{\text{tile}}[j]$ for each tile position.

MatE and VecU run concurrently, with VecU consuming the score tiles that MatE produces. The LSU's pre-compiled schedule explicitly orchestrates the cycle-by-cycle handoff. When the final context tile is consumed, O_i/l_i yields the exact attention output with no intermediate score tensor ever resident in SRAM.

After the last attention tile, `attn_out` resides in `activation_buffer`. The LSU then immediately dispatches `ISSUE_MAT_E attn_out_proj` — a 1,024 × 3,072 GEMM projecting the multi-head attention output back to the residual stream dimension, using the same weight-stationary dataflow as Stage 5.

13. Stage 11 — Feed-Forward Network (MatE + VecU)

Three dispatches in rapid succession constitute the feed-forward sublayer. Collectively they represent the dominant compute and bandwidth cost of every transformer layer.

#	Dispatch	Operation	Shape	Notes
1	`ISSUE_MAT_E ffn_up`	MatE: hidden → intermediate	3072 × 8192	∼2.67× expansion; largest GEMM
2	`ISSUE_VEC_U silu_mul`	VecU: SiLU activation gate	8192 elements	$\text{SiLU}(x) = x \cdot \sigma(x)$ via LUT
3	`ISSUE_MAT_E ffn_down`	MatE: intermediate → hidden	8192 × 3072	Weights stream from LPDDR

Table 3: FFN sublayer dispatch sequence (Llama-3.2-3B).

Each MatE dispatch pulls a fresh weight slice from LPDDR5X via the weight_stream_buffer at full 12 GB/s throughput. MatE runs at ∼50% utilization on average (weight-bound). VecU's SiLU pass processes 8,192 elements across 8 lanes in ∼50 μops, with the sigmoid computed via the 64-entry LUT in `codebook_const_rom`. The FFN sublayer accounts for roughly 70% of each layer's compute and 70% of each layer's LPDDR traffic.

14. Stage 12 — RMSNorm and Residual (VecU)

LSU dispatches `ISSUE_VEC_U rmsnorm_residual`. VecU reads the current layer's residual stream from `activation_buffer`, adds the FFN output, computes the RMS over the 3,072 elements, looks up $1/\sqrt{\text{RMS}}$ from the rsqrt LUT, multiplies through element-wise, and writes the result back. The microcoded sequence is ∼40 μops; the reciprocal-square-root lookup is the hot transcendental on this path.

`activation_buffer` now holds the normalized, residual-summed output of layer 0 — the input vector for layer 1. The LSU's program counter increments to the layer-1 entry point. Llama-3.2-3B has 28 layers; each costs ∼5 ms of LPDDR-bound time on this chip. Per-token latency: 28× ∼5 ms ≈ 134 ms/token → ∼7.4 tok/s. Execution loops back to Stage 1 with `layer = 1`.

15. Stage 13 — LM Head, Sampling, and Emission

After layer 27, LSU fires `ISSUE_MAT_E lm_head` — a 3,072 × 128,256 projection from the final hidden state to the full vocabulary logit space. This is MatE's largest single GEMM by output dimension, though the input is still a single token vector, so the weight-streaming overhead dominates compute. The LM head benefits from warm-cache behavior of the weight_stream_buffer, which retains frequently-used hot slices across tokens.

VecU runs sampling over the 128,256-element logit vector (∼30 μops): temperature scaling, top-k masking, top-p nucleus truncation, and a single softmax + categorical draw. The chosen token ID — a 17-bit integer — is placed in HIF's transmit buffer.

HIF packetizes the token as a USB-C 2.0 bulk transfer and delivers it to the host driver. The runtime (llama.cpp, a custom Python loop, or equivalent) receives the token ID and renders it on screen.

The LSU immediately resets its program counter to layer 0, seeds `activation_buffer` with the new token's embedding, and begins the next decode pass. The token-to-token loop runs ∼125–135 ms per token — 7–8 tok/s sustained.

16. Complete Chip Dataflow

```
HIF (boot) ---> LPDDR (weights at rest)
                                                    +-- prefetch (concurrent) --+
LSU -> MSC -> PHY -> LPDDR -> PHY -> MSC -> weight_stream_buffer -> MatE
    (xlate)         (DRAM read)     (data ret)   (SRAM staging)        |
                                                                        v
activation_buffer <-- VecU (RoPE) <-- MatE (QKV proj) --> KCE --> kv_scratchpad
                                                                        |
       MatE (output-stationary Q.K^T, compressed) <-- kv_scratchpad <--+
                                |
                                v
       VecU (online FA-3 softmax) --> MatE (softmax.V tiles) --> attn_out
                                                                        |
                                                                        v
       MatE (FFN up) --> VecU (SiLU) --> MatE (FFN down) --> residual
                                                                        |
                                                                        v
```

```
VecU (RMSNorm) --> activation_buffer -- (loop x 28 layers)
                                                          |
                                                          v
MatE (LM head) --> VecU (sample) --> HIF TX --> USB-C --> host
```

Eight blocks. One assembly line. Every cycle, somewhere on the chip, a multiplier is firing, a Hadamard butterfly is permuting, or a softmax running state is updating. The entire computation is choreographed by the LSU's pre-compiled microcode — no runtime decisions, no branch prediction, no surprises.

17. Performance Summary

Metric	Value	Notes
Sustained DRAM bandwidth	12 GB/s	LPDDR5X-8533 ×16 after overhead
Peak compute (MatE)	0.13 TOPS	128 GOPS INT8×INT4 @ 1 GHz
Decode throughput	7–8 tok/s	Llama-3.2-3B 134 ms/tok; Mistral-NeMo-3B 125 ms/tok
Typical decode power	2.4 W	Peak prefill 3.1 W; USB-C 15 W budget
KV compression ratio	3.0×	TurboQuant 3-bit; zero multipliers
Total die area	4 mm^2	2.0 × 2.0 mm; TSMC N16FFC
NRE cost (shuttle + IP)	\$140–260K	IMEC mini@sic 2.0; vs ∼\$500–700K flagship

Table 4: Lambda v2 performance and area summary.

Model	Params	W4 Size	ms/tok	tok/s	Status
Llama-3.2-1B	1.24 B	0.62 GB	51.9	19.3	Comfortable
Gemma-2-2B	2.0 B	1.00 GB	83.7	11.9	Comfortable
Mistral-NeMo-3B	3.0 B	1.50 GB	125.6	8.0	Primary demo
Llama-3.2-3B	3.21 B	1.61 GB	134.4	7.4	Interactive
Phi-3.5-mini	3.82 B	1.91 GB	159.9	6.3	Borderline
5B ceiling	5.0 B	2.50 GB	209.3	4.8	At threshold

Table 5: Model class coverage at Lambda v2 baseline operating point.

Lambda v2 Microarchitecture Dataflow Walkthrough. Companion to `Lambda_v2_4mm2.yaml`, `Lambda_v2_floorplan.` and `STATUS.md`. Numbers validated by `scripts/v2_design_space/lambda_v2_audit.py`.